OptiMAC: Adaptive Security Optimization for Message Authentication Code in Adversarial Environment

SeyedMohammad Kashani*, Erfan Khademnia†, Görkem Emirhüseyinoğlu†, Yilu Dong‡, Tianwei Wu‡,
Sang Wu Kim*, Ashfaq Khokhar*, Farid Nait-Abdesselam§, and Syed Hussain Rafiul‡,
*Department of Electrical and Computer Engineering, Iowa State University
† Industrial and Manufacturing Systems Engineering, Iowa State University
‡School of Electrical Engineering and Computer Science, Pennsylvania State University
§Laboratory of Informatics at Paris Descartes, Université Paris Cité
* {kashani,swkim, ashfaq}@iastate.edu, † {gorkem, erfank}@iastate.edu ‡ {yiludong,tvw5452,hussain1}@psu.edu
§ farid.nait-abdesselam@u-paris.fr

***Abstract*—Integrity verification utilizing Message Authentication Codes (MACs) can impose high overhead by generating and transmitting extra bits. Alternatively, grouping multiple messages and sending only one MAC for the group can reduce the computational and energy expenditure. However, in such grouping scenarios, the MAC depends on the each of the messages to validate the integrity of the entire group. Therefore, even with a single message loss, the MAC becomes unusable. In adversarial wireless environments, such as under jamming, this grouping strategy can make the system vulnerable to a denial-of-service attack. To address this, we propose OptiMAC, which addresses the shortcomings of the Progressive MAC (ProMAC) framework to increase the resiliency to message losses. In the ProMAC framework, each MAC is derived from an evolving state that holds a buffer of multiple messages and is updated with every new message. While the ProMAC framework's security is proven in principle, it lacks a systematic approach to defining the optimal state update rule for varying channel quality. Our proposed OptiMAC framework (i) formulates the state update process as a message grouping optimization problem and maximizes the number of usable MACs for any jamming probability, and (ii) uses simulations to find the optimum MAC length, which further maximizes throughput. Experiments over WiFi and 5G indicate improvements in security and efficiency.**

I. Introduction

Ensuring data integrity is fundamental to secure communication. Without integrity checks, receivers cannot verify the authenticity of received data, leaving systems vulnerable to malicious modification. The critical role of integrity verification has been underscored by real-world incidents; for example, in 2011 a U.S. drone reportedly lacking GPS integrity checks was captured after adversaries hijacked it by transmitting falsified GPS signals (a GPS spoofing attack) [9]. In practice, data integrity is most commonly realized via Message Authentication Codes (MACs): a short tag computed from the message using a secret key shared between sender and receiver. Transmitting and verifying MACs, however, introduces both communication and computational overhead, which is especially costly in power-constrained and latency-sensitive systems. Moreover, in the fifth-generation (5G) cellular standard, MAC deployment at certain layers is not mandatory to maintain high performance and is left optional for mobile network operators to implement [1], [27]. This further motivates techniques that reduce overhead without compromising security.

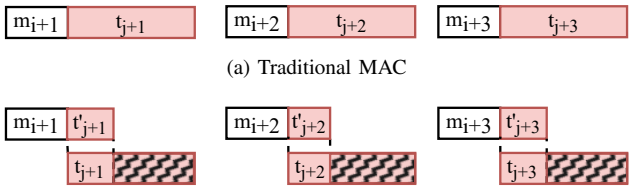


Fig. 1: 'm_i' represent the message and 't_j' represents the MAC (also called tag). Truncating the tag decreases the security [3], [22].

Prior attempts to cut overhead follow two main directions. The first is to shorten the tag itself ("short MAC," Fig. 1b). While this immediately saves bits, it also lowers the difficulty of successful forgery [3]; in fact, short MACs have been abused in practice to modify packets accepted by a badge reader and gain unauthorized access [22]. The second direction is to use one tag across multiple messages using aggregate MAC or aggregate message techniques [10], [19], [8]. Nonetheless, aggregating creates verification dependencies: the tag can only verify the integrity of the group if all of its dependent messages arrive correctly. In lossy or adversarial wireless settings (e.g., under jamming), losing even a single dependent message makes the tag unusable and enables denial-of-service against the integrity check [11], [12], [25], [26].

Armknecht et al. [3] proposed the Progressive MAC (ProMAC) framework, which generalizes many aggregation schemes by maintaining an internal state (a buffer) that is updated with each new message and then used to generate tags. A notable realization of this framework, the Whips scheme, cross-assigns tags across multiple messages so that each message is verified several times. This allows shorter tags to achieve the same effective security level as a longer tag can provide in a traditional MAC [3]. However, like aggregate MAC, Whips was designed for error-free channels. In practice, on jammed or lossy wireless links, Whip is susceptible to sandwich attack [25]. In adversarial settings, an integrity-check scheme must therefore explicitly account for message loss and

TABLE I: Prior approaches vs. OptiMAC (qualitative comparison).

Approach	Overhead handling	Jamming resistance	# tag assigned per msg	# tag (n) vs. # msg (m)	Tag size
Traditional MAC	None	Highest	1 (fixed)	$n = m$ (fixed)	High
Short MAC [22]	Tag truncation	Highest	1 (fixed)	$n = m$ (fixed)	Low
Agg. MAC [10], [15]	Grouping	Low	1 (fixed)	$n \leq m$ (semi-flexible)	High
Agg. msg [8], [19]	Grouping	Low	1 (fixed)	$n \leq m$ (semi-flexible)	High
ProMACs [3], [13]	Stateful multi-verification	Low	≥ 1 (flexible)	$n = m$ (fixed)	Low
SP-MAC [25]	Golomb rulers based dependency assignment	Medium	≥ 1 (flexible)	$n = m$ (fixed)	Low
OptiMAC	**Optimizing dependencies**	**High**	**Optimized**	$n \leq m$ **(fully flexible)**	**Optima**

maximize goodput, which jointly accounts for the amount of message bits that is integrity verified and the total transmitted tag bits. Intuitively, goodput improves when tag overhead is reduced while ensuring that a equal or higher number of message bits remain verifiable. Directly optimizing this trade-off, however, is extremely challenging.

To solve this problem, our proposed OptiMAC approach follows a divide-and-conquer strategy to improve the goodput by first maximizing the expected number of tag verification and second finding the optimal tag length.

Summery of contributions

Flexible integrity-check framework. We propose OptiMAC, an extension of the Progressive MAC (ProMAC) framework, to enable applications to choose flexible arbitrary tag-to-message ratios and fine-tune the balance between integrity overhead and security.

Optimization-based tag assignment. We designed a novel optimization model that systematically assigns tags to messages so as to maximize the expected number of usable tags under packet loss and jamming.

Precomputed methods for online adaptation. OptiMAC enables applications to precompute optimal configurations for a range of channel conditions and then adapt to the appropriate configuration during online operation.

Comprehensive evaluation. Through large-scale simulations and real-world live UDP webcam streams over Wi-Fi and 5G, we evaluated OptiMAC Across a range of loss and jamming conditions. We demonstrate that OptiMAC achieves superior goodput while maintaining a 256-bit security level, in particular, under periodic jamming.

Overall, OptiMAC unifies security, efficiency, resilience, and flexibility in a single framework, offering a practical and theoretically sound solution to integrity verification in lossy and adversarial communication environments.

II. Preliminaries

To establish the foundation of our proposed framework, we begin by defining the essential performance metrics, including the Security Level Requirements and Goodput.

Security Level Requirement

Many applications specify a required security level S (e.g., $S = 256$ bits) per message. If each usable tag verification contributes $|t|$ bits of tag strength, then a message meets the security level requirement when it accumulates enough verifications so that the cumulative strength reaches S.

A. Goodput Metric

We use *goodput* to quantify efficiency: the ratio of messages that are verified (received and met security level requirements) relative to the total bits transmitted (messages plus tags).

$$G = \frac{\sum \text{verified message bits}}{\sum \text{transmitted messages bits} + \sum \text{transmitted tag bits}}. \tag{1}$$

III. Literature Review

Various integrity verification methods have been developed to reduce communication and computational overhead. The overhead imposed by Message Authentication Codes (MACs) is a significant issue in bandwidth-limited networks such as CAN [19] and in high data rate applications within 5G networks [1]. Truncated MACs have been proposed as a practical solution to mitigate this overhead. However, truncation comes at the expense of security strength, making the system more vulnerable to collision attacks [3].

Other overhead-reducing schemes, such as aggregate and compound MACs, minimize computational and communication overhead by verifying multiple messages with a single tag. While this approach eliminates the need to compute and transmit a MAC for every individual message, it introduces a critical weakness: the loss or corruption of any single packet within the group prevents the integrity verification of the entire set. This weakness arises because all messages contribute to the generation of the common MAC, making them mutually dependent. Consequently, the integrity check fails if even one message is missing, leaving the common MAC unusable.

To address this weakness, it is important to recognize that aggregate and compound MAC schemes were originally designed for error-free communication links, which makes them particularly susceptible to packet losses. Such losses may arise naturally in lossy channels or be deliberately introduced through jamming attacks, indicating the presence of an adversary [23], [12], [25], [11]. Detecting and preventing jamming is especially challenging in wireless environments. Random jamming, for example, mimics the behavior of a naturally lossy channel, making reliable detection extremely difficult [23]. Therefore, MAC overhead-reducing schemes must explicitly consider the impact of various jamming strategies, including

random and periodic jamming, to ensure resilience under adversarial conditions. In this context, assessing the message-to-tag dependencies becomes essential to maintain the integrity and availability of the system in the presence of a jammer. Understanding these dependencies is vital for designing robust integrity verification schemes that can withstand packet losses and malicious attacks.

A. Problem Background

The earliest integrity-check overhead reduction approach can be traced to [4], which introduced an XOR-based aggregation technique that combines multiple digital signatures into a single compact signature. This method not only reduces the total size of transmitted signatures but also provides provable security guarantees[5], [4], [12]. Building on this idea, [10] has applied the aggregation technique to MAC schemes, combining multiple tags into a single tag to reduce communication overhead. An alternative direction, explored in [15], [14], [16], proposed sequential aggregation, where each new signature is computed over the current message concatenated with the preceding signature, effectively chaining multiple signatures into one.

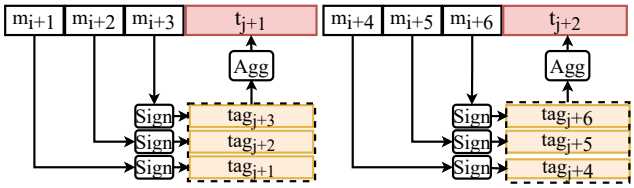


Fig. 2: Aggregate MAC [10]

Figure 2 illustrates the MAC aggregation technique proposed in [10]. Alternative approach, as shown in Figure 2, involves aggregating messages to generate a single tag, transmitting the resulting tag for all aggregated messages [19], [6], [8], [24]. The 'Sign' operation indicates the MAC generation

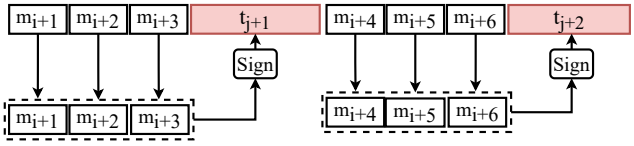


Fig. 3: Aggregate Messages [8]

process, and 'Agg' depicts the aggregation step. While these two schemes are different in nature, from the message and MAC interdependency point of view, they behave the same. Thus, from now on, the aggregate MAC and aggregate message terminology might be used interchangeably.

In [3], the Progressive MACs framework is introduced, which provides a novel approach to generalize all the previous MAC aggregation schemes. Progressive MACs framework utilizes an internal state mechanism that is updated with every new message and a tag generation mechanism that inputs from the internal state. Figure 4 illustrates one possible realization of a progressive MAC framework, called Whips, with an internal state of size three. As shown in Figure 4, the message

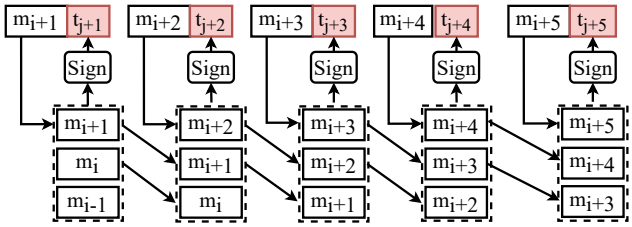


Fig. 4: ProMACs [3], [13] cross-assigns message to tag. This Figure is reproduced from [25].

m_{i+1} is involved in the generation of tags t_{j+1}, t_{j+2}, and t_{j+3}. This way, the message m_{i+1} will be verified by the three tags, which means to successfully modify m_{i+1}, an attacker would need to guess all three tags correctly, equivalent in security to using a tag three times as large. For instance, if a message requires the security level of a 256-bit tag, a Progressive MAC with a state of size three could achieve this with 86-bit tags, as each message is verified three times. This state mechanism allows Whips to gradually verify the integrity of packets as more tags are received, effectively increasing the security level over time without requiring full-length tags for each packet [3].

Therefore, Progressive MAC-based schemes [3], [25], [13], [24] are not confined to fixed-length tag configurations. Depending on security requirements and bandwidth constraints, they can employ truncated tags that, through cumulative verification across multiple packets, offer security guarantees comparable to traditional MACs [3]. By tuning parameters such as tag length, state-update rules, and internal state size, the Progressive MAC framework can adjust to application constraints [3]. However, despite this versatility, the ProMAC framework lacks a systematic method for determining optimal parameter choices for different application constraints. Without a systematic approach, designing a good-performing system becomes extremely difficult. This is even more critical in adversarial environments, where periodic or random packet loss can severely attack the integrity check scheme.

To address this limitation, we propose OptiMAC, a graph-based mapping approach that enables a systematic design and analysis of the ProMAC framework. This graph-based method systematically analyzes and adjusts dependencies between messages and tags, resulting in an optimized structure that maximizes the number of verifiable packets under conditions of high packet loss, achieving optimal security tailored to specific channel conditions.

B. Limitations in Adversarial Environments

The limitations of MAC or message aggregation schemes in adversarial scenarios have been previously pointed out. Kolesnikov et al. [11] demonstrated that MAC overhead-reducing schemes can increase the system's vulnerability to DoS attacks. To address this, Kolesnikov et al. proposed a "shifted bitwise XOR" approach to mitigate the impact of packet loss on the verification of other messages [11].

Wagner et al. [25], introduce SP-MAC, a resilience-oriented authentication scheme extending the Progressive MAC (ProMAC) framework with a combinatorial construction based on Golomb rulers and generalized g-Sidon sets [25]. This dependency structure is designed to limit the effect of a message loss on the other messages, making it more resilient in adversarial scenarios. Despite the SP-MAC claim of having the *optimal* dependency distribution, it lacks quantifiable objectives. The provable guarantees are controllable latency and controllable effect of message-loss on other messages. Though this doesn't by it self guarentee optimal latency or optimal goodput of the overall system. Moreover, Similar to many other schemes, the number of tags that are required to be genereted equals to the number of messages, whereas aggregate MAC and Our proposed OptiMAC allow flexibility on the number of tag used per message, reducing computational overhead.

It is evident from [25], [11], [26] that grouping messages or aggregating tags to reduce overhead introduces verification dependencies between messages. These dependencies can add latency and make the system vulnerable to jamming. While some prior work [11], [25] has recognized this problem and proposed heuristic solutions, our study presents an optimization approach to find the optimal parameters in adversarial environments.

In summary, progressive MAC introduces a framework that generalizes all MAC aggregation schemes through an evolving internal state and an arbitrary update and tag generation function. However, the lack of guidance to find the optimum settings for this framework, especially in challenging under-attack environments, leaves a gap that we are addressing in this paper. Our proposed OptiMAC framework maps the state and the update functions of the ProMAC framework to a graph, providing a systematic approach to analyze and optimize any MAC aggregation schemes. Addressing this issue is critical for improving the efficiency and security of communication systems, particularly in adversarial wireless environments.

IV. Threat Model

In this work, we consider an adversarial setting modeled by a simplified version of the Dolev-Yao attacker model [7]. The attacker has the capability to inject, drop, or modify packets transmitted over the network. Specifically, the attacker can introduce forged packets into the communication channel with the intent of deceiving the receiver or disrupting the communication protocol. The attacker can also intercept and remove packets from the communication channel, preventing their delivery to the intended recipient. Additionally, the attacker can alter the content of transmitted packets, potentially corrupting the data or causing the receiver to accept false information.

We assume that the attacker does not possess the cryptographic keys used for securing the communication. Consequently, the attacker cannot decrypt encrypted messages or generate valid encrypted messages and authentication tags. The inability to decrypt or correctly encrypt messages limits the attacker to manipulate the transmission medium and observable packet structures without understanding or creating valid encrypted content.

V. Design Goals

The security of integrity checks is fundamentally dependent on the length of the MAC employed; this is also referred to as the security level in [3]. In the conventional approach, enhancing security involves utilizing larger tags, which incurs additional overhead. Instead of merely increasing the length or number of tags, we focus on optimizing tag-to-message dependencies to effectively enhance performance and security without increasing the actual tag length or the total number of tags. The followings will summarize our design goals.

G1: High Goodput. A high-goodput system maximizes the message bits that have been verified above the required security threshold while at the same time minimizing the total tag bits transmitted.

G2: Meeting Security Level Requirements. Increasing the number of tag bits per message results in an exponential enhancement of integrity check security [3]. Increasing the tag length above some threshold would be unnecessary for practical application and would have a reverse effect on the goodput performance. Thus, the tag length must be optimized in a way that maximizes the goodput while maintaining the security requirements.

G3: Systematic, Adversary-Aware Optimization. A desirable MAC scheme should offer a principled, model-driven procedure, rather than heuristic tuning, for operation under lossy and adversarial conditions. Given fixed operating parameters (number of messages, number of tags, and a loss/jamming probability), it should systematically compute the optimum message-to-tag assignment, and thereafter finds the optimum tag length to maximizes the goodput of the system.

G4: Flexible Tag-to-Message Ratio. The system should support arbitrary effective average tags per message (e.g., $2/3$), rather than only the reciprocal ratios ($1/g$) imposed by rigid grouping of the Aggregate MAC.

VI. Dependency Mapping

Grouping messages or aggregating tags introduces dependencies in the verification process, making the system more vulnerable to jamming attacks. By constructing the dependency graph, we model the relationships between messages and tags.

Dependency Graph

Assuming the general case of a system with 'm' messages and 'n' tags where:

$$\mathbf{m} = \{m_1, m_2, \cdots, m_i, \cdots, m_m\}$$
$$\mathbf{t} = \{t_1, t_2, \cdots, t_j, \cdots, t_n\}$$

We define the dependency graph such that $DG = (\mathbf{V}, \mathbf{E})$ is an undirected graph representing the tag-to-message dependencies. DG is defined as follows:

$$\mathbf{V}_1 = \{m_i \mid i \in \{1, \cdots, m\}\},$$

is the set of vertices, where each m_i represents a message.

$$\mathbf{V}_2 = \{t_j \mid j \in \{1, \cdots, n\}\},$$

is the set of vertices, where each t_j represents a tag, And

$$\mathbf{V} = \mathbf{V}_1 \cup \mathbf{V}_2.$$

$$\mathbf{E} \subseteq \{(u, v) \mid u \in \mathbf{V}_1, v \in \mathbf{V}_2\},$$

is the set of undirected edges, where an edge $\{m_i, t_j\} \in \mathbf{E}$ signifies that tag t_j verifies message m_i.

Each vertex $v \in \mathbf{V}$ can be a message or tag, and each edge $e \in \mathbf{E}$ indicates a tag-to-message assignment. Figure 5 visualizes the dependency graph for the progressive MAC.

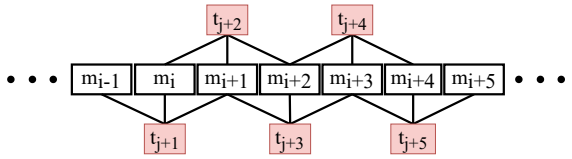


Fig. 5: Whips dependency graph with internal state size of 3.

Dependency Matrix (X)

Based on the dependency graph, we define the dependency matrix X, where the rows correspond to messages and the columns correspond to tags. The entries of the matrix are defined as follows:

$$X_{i,j} = \begin{cases} 1 & \text{if message } m_i \text{ is assigned to tag } t_j \\ 0 & \text{otherwise} \end{cases} \tag{2}$$

For example, the dependency matrix of the Whips scheme, as shown in Figure 5, is as follows:

$$\mathbf{X} = \begin{array}{c|cccccccc} & t_1 & \cdots & t_{j+1} & t_{j+2} & t_{j+3} & \cdots & t_n \\ \hline m_1 & 1 & \cdots & 0 & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ m_{i-1} & 0 & \cdots & 1 & 0 & 0 & \cdots & 0 \\ m_i & 0 & \cdots & 1 & 1 & 0 & \cdots & 0 \\ m_{i+1} & 0 & \cdots & 1 & 1 & 1 & \cdots & 0 \\ m_{i+2} & 0 & \cdots & 0 & 1 & 1 & \cdots & 0 \\ m_{i+3} & 0 & \cdots & 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ m_m & 0 & \cdots & 0 & 0 & 0 & \cdots & 1 \end{array}$$

Status Vector

Let $\mathbf{M}$ and $\mathbf{T}$ denotes the status vector, where each entry indicates if the message or tag is correctly received or lost (modified or jammed). $\mathbf{M}$ is defined as follows:

$$\mathbf{M} = \begin{bmatrix} M_1 & M_2 & M_3 & \cdots & M_i & \cdots & M_m \end{bmatrix}$$

Where $M_i = 1$ if message m_i is successfully received, and $M_i = 0$ otherwise. The tag status vector $\mathbf{T}$ is defined as follows:

$$\mathbf{T} = \begin{bmatrix} T_1 & T_2 & T_3 & \cdots & T_j & \cdots & T_n \end{bmatrix}$$

Where $T_j = 1$ if tag t_j is successfully received, and $T_j = 0$ otherwise.

Tag Usability (U)

A tag is usable if the tag and all the dependent messages are correctly received. Let $U_j = 1$ if tag t_j is usable, and $U_j = 0$ otherwise. A usable tag can authenticate all the messages that are dependent on it, whereas an unusable tag can not verify any messages. the U_j in the tag usability vector $\mathbf{U} = \{U_1, ..., U_n\}$ is given by:

$$U_j = T_j \cdot \prod_{i \in \{i \mid X_{ij}=1\}} M_i \tag{3}$$

Authentication (A)

Let A_i denote the number of usable tags verifying message m_i. The A_i from the authentication vector $\mathbf{A} = \{A_1, ..., A_m\}$ is as follows:

$$A_i = \sum_{j \in \{j \mid X_{ij}=1\}} U_j \tag{4}$$

For example, for the the Whips in Fig. 4, if all tags and messages are correctly received, vector A from $i-1$ to $i+5$ will be as follows:

$$\mathbf{A} = [\cdots, 3, 3, 3, 3, 3, 3, 3, \cdots] \tag{5}$$

However, in the case that message m_{i+2} is lost, tags t_{j+2}, t_{j+3}, and t_{j+4} become unusable. Therefore, vector A will be as follows:

$$\mathbf{A} = [\cdots, 3, 2, 1, 0, 1, 2, 3, \cdots] \tag{6}$$

To calculate the expected authentication vector, the binary status of messages and tags is substituted with the probability that they are successfully received. Let P_i be the probability that message m_i is successfully received, and let Q_j be the probability that tag t_j is successfully received. The probability that tag U_j is usable is as follows:

$$Pr(U_j = 1) = Q_j \cdot \prod_{i \in \{i \mid X_{ij}=1\}} P_i, \tag{7}$$

then the expected number of usable tags for a given message m_i can be expressed as:

$$E[A_i] = \sum_{j \in \{j \mid X_{ij}=1\}} U_j \cdot Pr(U_j = 1) \tag{8}$$

Where $U_j = 1$ represents a usable tag and $U_j = 0$ otherwise.

This probabilistic formulation allows for the evaluation of the authentication process under varying channel conditions, providing the necessary cornerstone for optimizing the tag-to-message assignments to enhance security, reliability, and performance.

VII. Optimization Model

Our proposed scheme, OptiMAC, extends the Progressive MAC (ProMAC) framework with an optimization layer that determines how tags are generated, distributed, and assigned to messages.

TABLE II: Nomenclature for the OptiMAC's optimization model

Sets	Description
I	Set of message indices $\{1, \ldots, m\}$
J	Set of tags indices $\{1, \ldots, n\}$
K	Set of possible number of messages that can be assigned to a tag $\{0, \ldots, m\}$
Decision Variables	Description
x_{ij}	Binary variables representing message assignments to tag. Equal to 1 if message i is assigned to tag j.
z_j^k	Binary variables representing the total number of messages assigned to tag j. Equal to 1 if k messages are assigned to tag j.
y_{ij}^k	Binary variables representing the combination of a message, its tag, and the total count of messages assigned to that tag. Equal to 1 if message i is assigned to tag j, and there are k messages assigned to that tag.
U_{ij}^k	Expected value of tag j being usable for message i if message i is assigned to tag j, and there are k messages assigned to tag j.
A'	Dummy variable for equally representing the expected number of usable tags across all the messages
Parameters	Description
$w^k(p, q)$	Pre-calculated authentication probabilities with respect to p, q for a tag with k number of assigned messages
p	Success probability of each message
q	Success probability of each tag

Maximizing the $E[A]$ ensures goals G1 and G2. Therefore, we have modeled this problem as follows:

$$\max \quad \sum_{i\in I}\sum_{j\in J}\sum_{k\in K} U_{ij}^k \tag{9}$$

$$s.t.$$

$$w^k y_{ij}^k |t_j| = U_{ij}^k \qquad \forall i \in I, j \in J, k \in K \tag{10}$$

$$\sum_{i\in I} x_{ij} = \sum_{k\in K} k z_j^k \qquad \forall j \in J \tag{11}$$

$$\sum_{k\in K} z_j^k \leq 1 \qquad \forall k \in K \tag{12}$$

$$x_{ij} + z_j^k \geq 2y_{ij}^k \qquad \forall i \in I, j \in J, k \in K \tag{13}$$

$$\sum_{j\in J}\sum_{k\in K} U_{ij}^k = A' \qquad \forall i \in I \tag{14}$$

$$\sum_{j\in J} x_{ij} \geq 1 \qquad \forall i \in I \tag{15}$$

$$A' \geq 0 \tag{16}$$

$$x_{ij}, z_j^k, y_{ij}^k \in \{0, 1\} \tag{17}$$

The objective function is to maximize the expected number of usable tags for all messages (9). The expected tag usability of each message for its respective tag when in total k messages are assigned to that tag is calculated in equation (10). Equations (11) and (12) ensure the total number of messages assigned to each tag is represented accurately, while equation (13) handles the assignments for each message, tag, and assignment combinations. During our test runs, we noticed that the expected number of usable tags is not equally distributed. To ensure the equal distribution, equation (14) is incorporated. Finally, equation (15) makes sure each message is assigned to at least one tag. The remaining constraints are the binary restrictions on the decision variables.

Equation (8) is highly non-linear and requires linearization to be used in MILP formulation. Let w_{ij}^k be the pre-calculated authentication probability values for all possible message-to-tag assignments, where the superscript k represents the number of messages assigned to tag j.

Note that since the usability vector is a binary random variable where $U_j \in \{0, 1\}$, the probabilities and expected values are equivalent and these terms can be used interchangeably for clarity. Let U_{ij}^k represent the probability that tag t_j is usable for message m_i when a total of k messages are assigned to tag t_j. By maximizing the summation of U_{ij}^k over j, i, k, we maximize the expected number of usable tags for all messages.

In this model, y_{ij}^k will be set to one to assign the correct pre-calculated w_{ij}^k to U_{ij}^k. While the introduction of the variable k linearizes the model, it increases computational complexity and runtime for larger problems. To solve this mathematical model, Gurobi version 11.0.2 is utilized as a baseline solver on default settings, and it was implemented in Python version 3.9.6. The optimization model is available in [2].

Through this formulation, OptiMAC directly addresses the four design goals. For G1 and G2 (High Goodput and Security Level Requirement), the optimization ensures that more messages are verified by a sufficient number of tags meeting the security requirement. For G4 (Flexibility), the MILP framework allows the system to adjust the number of tags, their sizes, and their assignment strategy dynamically, depending on measured channel conditions and application demands. And finally our approach is systematic and provides an optimum solution for different adversary condition, achieving G3 (Systematic Adversary-Aware Optimization).

In summary, OptiMAC unifies security, efficiency, resilience, and flexibility in a single framework. By discarding the traditional "one message, one tag" rule and by leveraging optimization techniques, the scheme maximizes the expected number of usable tags and guarantees robust integrity verification under both random and adversarial losses.

VIII. Security Analysis

The security foundation of the optiMAC builds upon the methodologies established in the Progressive MAC framework as discussed in Armknecht et al. [3]. The formal security properties and resilience of progressive MAC have been rigorously analyzed, providing strong guarantees against various attack

vectors, including forgery and replay attacks in high data rate, resource-limited environments. Additionally, the security of aggregated MAC techniques was validated in the work by Boneh et al. [5], which provides theoretical backing for the aggregation of MAC tags while preserving authentication and integrity.

IX. Performance Evaluation

We assess the performance of our scheme using the metrics defined in the following. We conduct a comprehensive analysis across both WiFi and 5G environments using a real-world testbed to capture the practical implications of our approach. By evaluating optiMAC under realistic conditions, we aim to demonstrate its effectiveness and resilience in low-latency WiFi settings and high-throughput 5G networks. This dual-network evaluation allows us to rigorously test our scheme's adaptability and efficiency across diverse conditions, validating its suitability for modern wireless communication scenarios.

A. Evaluation Metrics

1) Tag to Message Ratio (TMR): MAC overhead-reducing schemes utilize different amounts of energy and computation power, usually depending on the size and the number of tags they generate or transmit per message. However, comparing different schemes' efficiency is difficult, and to simplify comparison, we assume these schemes deploy the same algorithm and generate the same tag size. Based on this assumption, We define TMR ($\frac{n}{m}$), a metric designed to evaluate the efficiency of integrity check schemes, given by:

$$\frac{n}{m} = \frac{\text{total number of tags}}{\text{total number of messages}} \tag{18}$$

Assuming that transmitting number of tags is less than or equal to the number of messages, $\frac{n}{m}$ provides a normalized value between 0 and 1, where 0 indicates the optimal scenario where no tags are used, representing the highest possible efficiency and 1 indicates that the scheme has no improvement over the traditional method (one tag per message).

2) Tag-bits per Message (TbpM): Although Goodput shows the bandwidth efficiency of the MAC schemes, it does not capture the system's security strength. For example, in the Whips shown in Figure 4, for every message to achieve a security equal to 258-bit, three tags of size 86 bits are required. However, A packet loss, as shown in equation (6), resulted in 9 fewer verifications than expected, equivalent to 774 fewer bits of security overall.

Therefore, we define Tag-bits per Message as follows:

$$TbpM = \frac{\sum_{i=1}^{m} \sum_{j \in \{j|X_{ij}=1\}} Pr(U_j = 1)|t_j|}{m} \tag{19}$$

Assuming that $|t_j| = |t| \quad \forall j$ we can further simplify the equation above:

$$TbpM = \frac{|t| \sum_{i=1}^{m} E[A_i]}{m}, \tag{20}$$

where the tag length is represented by $|t_j|$, and there are in total of m messages. Equation (19), is the expected number of usable tag bits per message for statistical analysis, however for deterministic analysis , the $Pr(U_j = 1)$ must be replaced by U_j.

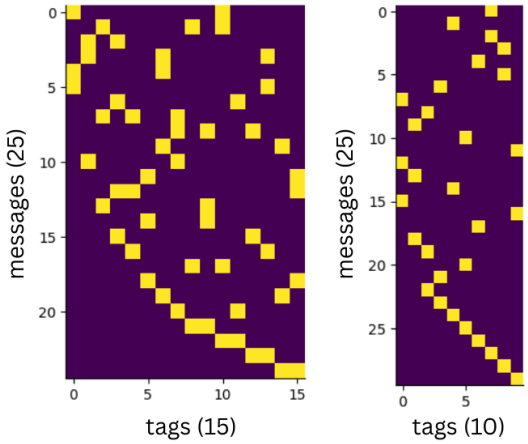


Fig. 6: Two Optimized Matrix **X** with TMR = 10/25 and 15/25 from left to right, respectability. The yellow squares indicate a 1 representing a message to a tag assignment.

3) Authentication Latency (L): Packet latency is when a packet is sent and when the integrity check scheme verifies it. Multiple factors influence it, including implementation details, data rates, and computational power. Additionally, the latency can vary depending on the network layer where the integrity check is deployed. Our objective is to specifically assess the impact of the integrity check scheme on latency compared to traditional MACs under both ideal and attack scenarios. It's important to note that underlying layers—such as the physical, data link, and transport layers—can also contribute to latency, especially when the system is under attack.

To facilitate a relative comparison with traditional MACs, we assume that all tags and messages are correctly received. Under this assumption, authentication latency is the number of additional messages that must be received before a specific message can be authenticated, assuming messages are transmitted sequentially. For instance, if tag t_j verifies messages $m_{i+1}, m_{i+3}, m_{i+7}$ then m_{i+1} can be verified only after $m_{i+\{2,\cdots,7\}}$ are received, which by our definition, the latency is six.

Let's assume that all the messages are sent in order based on their index. For example, m_1 is sent first and m_2 is sent afterward. We define the Tag Maximum Index (**TMI**) vector, where the j-th entry is the maximum index of the message that the tag t_j is verifying.

$$TMI_j = \max_{i \in \{i|X_{ij}=1\}} i \tag{21}$$

For example, in the compound MAC shown in Fig. 3, TMI_{j+2} corresponding to tag t_{j+2} equals $i + 6$, since the the messages that are verified by tag t_{j+2} are m_{i+4}, m_{i+5}, and m_{i+6}. The authentication latency vector **L**

is defined as follows:

$$L_i = \begin{cases} \left(\min_{j\in\{j|X_{ij}=1\}} TMI_j\right) - i & \text{if } A_i > 0 \\ \text{penalty} & \text{otherwise} \end{cases} \quad (22)$$

The penalty is the value assigned if message m_i is not verified by any tag. For example, the latency vector for the Progressive MAC is all zeros. However, the latency vector for the Compound MAC will be as follows:

$$L = [2, 1, 0, 2, 1, 0, \cdots]$$

4) Strength Number (SN): The Strength number SN represents the smallest ratio of messages or tags that, if lost, would prevent the verification of all messages. An upper bound for the strength number is the total number of tags divided by the number of messages, as destroying all tags would result in no verifiable message. If the set consists only of tags, it must include all tags to ensure no message can be verified. However, any tag within this set can be replaced by a message it verifies. If tag-to-message substitutions are selected strategically in a way that for at least two tags, a common message is replaced, the resulting set can potentially have a cardinality less than the number of tags.

Thus, the strength number can be formulated as the minimum number of messages that will make all the tags unusable divided by the number of messages. As a result, let $\mathbf{S} \subseteq \{m_1, m_2, \ldots, m_m\}$ and for any tag, there exists a message in SN that is verified by that tag.

$$\mathbf{S} = \{m_i \mid \quad \forall j, \exists i \quad \text{such that} \quad X_{ij} = 1\} \quad (23)$$

The $|\mathbf{S}|$ must be minimized to find the strength number. For a given matrix X, finding the strength number is similar to the classical set covering the problem with the following formulation. We define a binary variable s_i for each message.

$$s_i = \begin{cases} 1 & \text{if message } i \text{ is selected,} \\ 0 & \text{otherwise.} \end{cases} \quad (24)$$

The mathematical formulation of this set covering problem will be as follows:

$$\min \sum_{i=1}^{m} s_i \quad (25)$$

subject to

$$\sum_{i\in\{i|X_{ij}=1\}} s_i \geq 1, \quad \forall j \in \{1, 2, \ldots, n\} \quad (26)$$

$$s_i \in \{0, 1\}, \quad \forall i \in \{1, 2, \ldots, m\} \quad (27)$$

In this formulation, the constraint (26) guarantees no message will be verified. The objective function is of type cardinality, effectively counting the minimum possible number of messages that guarantee the total failure of the system. Note that the constraint (26) should only be used for the active tags in X. Further, $|S|$ must be normalized by the number of messages. Therefore SN is given by:

$$SN = \frac{\min(|S|)}{m}, \quad (28)$$

where m is the number of messages. The strength number effectively measures the security in terms of availability, representing the percentage of packet loss that results in system failure. Figure 7 provides a comparative analysis of the effects

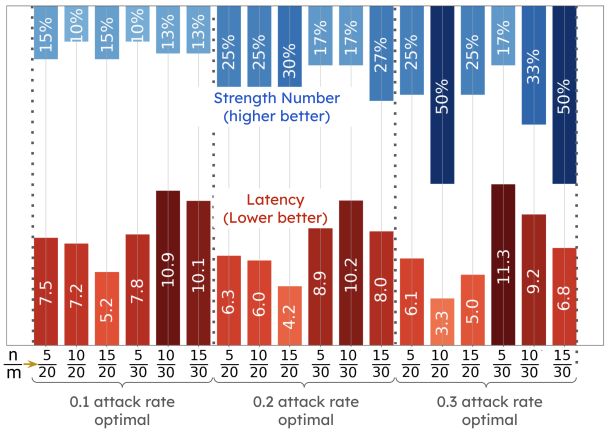


Fig. 7: Strength Number and Latency for the optimal solution by attack rates and Tag-to-Message Ratio ($\frac{n}{m}$).

of the optimization model on Strength Number (SN) and Latency (L) across different configurations optimized for three distinct attack rates: 0.1, 0.2, and 0.3. Each section corresponds to a specific attack rate and displays bars representing the values of SN (in blue) and L (in red) for various schemes, where each scheme is characterized by a different TMR n/m.

B. Real-world Testbed

We have created two real-world testbeds to evaluate the impact of packet interference on webcam video streaming using the UDP protocol on WiFi and 5G networks. The interference sources included an attacker modifying packet data. The objective is to assess the impact on transmission integrity and performance metrics, particularly under adverse conditions.

1) WiFi Testbed Setup: The experimental setup involved a host machine connected to a Logitech HD Pro C920 X webcam, which captured real-time video frames at a resolution 1080p and a frame rate of 30 fps. These frames were transmitted over a WiFi network using the UDP protocol. The UDP packets were customized by appending additional tags according to matrix $\mathbf{X}$.

The transmitter was equipped with an Intel WiFi 6 AX200 (rev 1a) adapter, configured on a Linux Ubuntu 24.04 LTS system with an Intel Core i7-11700K processor. The WiFi hotspot was managed using the nmcli tool, configured to operate on the 802.11 bg at 2.4 GHz band. The video data was encoded using JPEG Progressive format, with varying quality settings 25% corresponding to data 650 KB/s rate.

2) 5G Testbed Setup: We set up the 5G testbed using an open-source 5G software stack. We use the User Equipment (UE) and gNodeB implementation from OpenAirInterface [20] and the 5G Core Network from Open5GS [21].

The experiment runs on two host machines. Both machines run Ubuntu 22.04 LTS with Intel Core i9-14900K and 64 GB

of memory. Host 1 is equipped with 1 USRP B210 software-defined radio and runs the UE 1. Host 2 is equipped with 2 USRP B210s and runs the gNodeB, UE 2, and the 5G Core.

In our setup, Host 1 acts as the sender. It captures real-time video frames from a Logitech C930e webcam and sends the UDP packets to UE software. The UE 1 running on the same machine receives the packets, processes them in the 5G stack, and sends the data over-the-air to the gNodeB. The gNodeB in Host 2 receives the data from UE 1 and sends it to the Core Network for routing to UE 2. Then, the Core Network routes the packets from the gNodeB over-the-air to the UE 2. Our receiver listening on UE 2 finally receives the transmitted data frame.

3) Transmission Setup: The transmitted data was structured at the application layer with a custom frame format. Each frame included a 4-byte sequence number, an 8-byte timestamp, and a variable-length data field.

TABLE III: Experiment parameters for WiFi and 5G

	Data Rate (KB/S)	Message Size (bits)	Tag length (bits)
5G	300	768	512
WiFi	600	128	256

The data was then transmitted using the UDP/IP transport layer, to avoid further influence of the transportation layer. The details of the setup configuration for each setting are as follows. To accommodate flexibility to perform integrity checks for any given matrix $\mathbf{X}$, an increase in the complexity of the implementation was necessary. This complexity arose from the need to manage multiple packets before the integrity check was possible, which was achieved by utilizing a book-like buffer structure. In this structure, each "page" holds a certain number of packets based on matrix $\mathbf{X}$ dimensions. Once a page is full, it is processed and then released. Although this approach required a larger buffer and introduced additional complexity, we were able to demonstrate that it was manageable by successfully implementing and testing live video streaming at different data rates both in the WiFi and 5G testbed.

The impact of an attacker modifying packets on video transmission was assessed using several key metrics: the number of bits verified per message ($TbpM$), and goodput (G). Accurately measuring latency posed a challenge due to the need for precise synchronization between the two devices; therefore, we acknowledge that the latency measurements was not entirely accurate. However, in the future we will try to incorporate a mechanism to reliably measure the latency.

X. Results

In this section, we analyze the trade-off between security, goodput, and latency achieved by various MAC schemes based on their tag sizes and tag-to-message ratios (TMR). The tag-bits per message (TbpM) is a metric used to measure the integrity and availability of the system. Moreover, Goodput and TMR are metrics to show both the bandwidth and energy efficiency of the system..

A. Computation and Latency Overhead

To evaluate latency and computational overhead of different MAC schemes, we use an application-level metric based on real-time video streaming. Specifically, we measure the number of successfully verified frames per second (FPS) in our 5G experimental testbed, as shown in Fig. 8. Since the camera generates frames at a fixed rate of 30 FPS, the achieved FPS reflects the system's ability to process, authenticate, and verify packets in real time. Therefore, FPS serves as a practical proxy for both computational overhead and end-to-end latency.

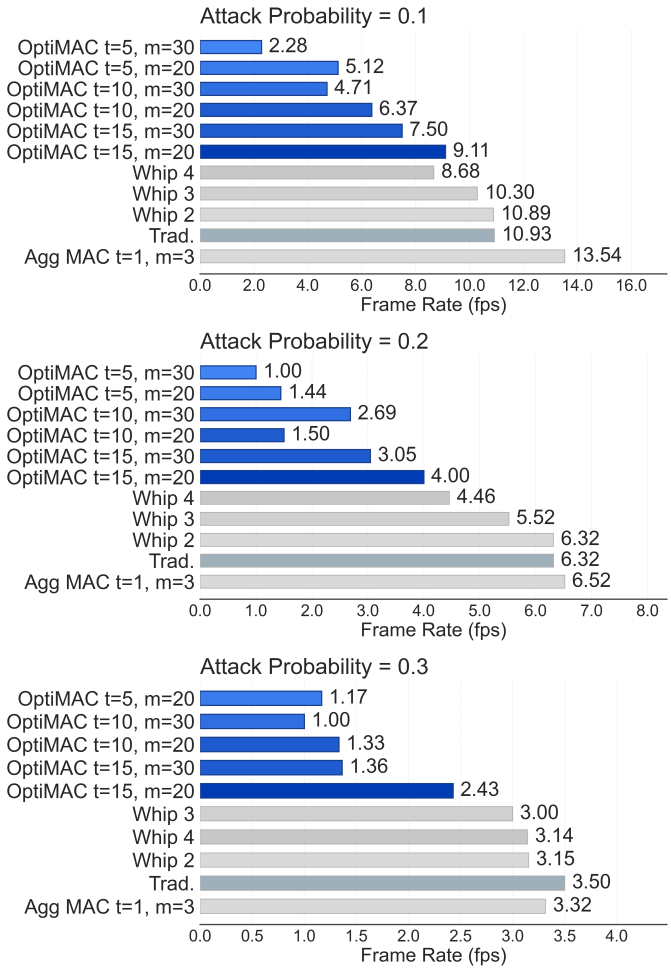


Fig. 8: Frame rate (FPS) under varying attack probabilities

Fig. 8 shows the achieved frame rates under different attack probabilities (0.1, 0.2, and 0.3). As the attack probability increases, all schemes exhibit a reduction in FPS. This degradation is primarily due to increased packet loss, which limits the number of successfully verified packets available for frame reconstruction. In our implementation correct decoding requires the availability of critical header and payload data. When a sufficient number of packets cannot be verified, the

decoder fails to reconstruct the frame, resulting in dropped frames and a corresponding reduction in FPS. Furthermore, as the dependency matrix size increases, the authentication process becomes more complex, leading to higher authentication latency (L). This effect is evident when comparing different OptiMAC configurations, where smaller matrices (e.g., t=10, m=20) achieve higher FPS than larger ones (e.g., t=15, m=30) due to reduced verification delay. Therefore, the observed decrease in FPS reflects not only increased computational and communication overhead, but also the impact of both insufficient verified data and higher authentication latency under adversarial conditions.

Our prototype is implemented in Python at the application layer and is not optimized for performance. Nevertheless, OptiMAC supports real-time streaming at practical frame rates. A production-level implementation (e.g., in C/C++ or at the data link layer) is expected to further reduce overhead and latency.

B. Tag-bit per Message (TbpM) vs Goodput

Similar to progressive MAC [3], adjusting the tag bits per message could be advantageous in balancing the overhead trade-off. A higher TbpM allows for shortening the tag length while satisfying the security requirements. However, in lossy channels, where the expected number of verifications is lower than in error-free channels, shortening the tag must be done carefully. For a MAC scheme to be considered secure, it must maintain a tag size of at least 64 bits, as per NIST standards [18]. However, NIST emphasizes the actual tag length that may vary depending on the specific algorithm employed, and some applications, like IPsec, mandate a minimum of 128-bit tags without truncation to ensure robust security [17]. Without loss of generality, In the rest of this paper, we assume more than 128 and 256 Tag-bits per Message (TbpM) are high security for WiFi and 5G, respectively.

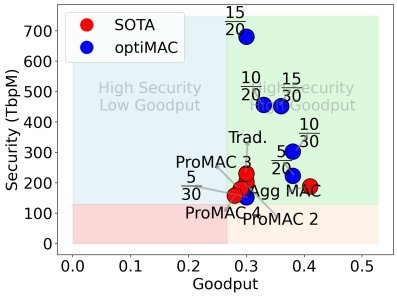


Fig. 9: TbpM vs. Goodput for WiFi where a 256-bit tag was used (above 128 bits is considered as high security) under 0.1 attack rate.

In our study, any scheme with TMR exceeding the threshold set by the aggregate MAC scheme (i.e., one tag per two messages) is a high TMR scheme. Low TMR schemes are expected to offer more efficient bandwidth and computational usage by lowering the number of tags transmitted, though this may impact security if not managed carefully. By comparing various approaches in terms of security (measured in TbpM), we demonstrate the potential of optiMAC schemes to balance these factors, particularly in adversarial environments.

The results, visualized in Figure 9, illustrate that optiMAC (indicated in blue) generally outperforms state-of-the-art methods (in red) by achieving both high security and goodput. The figure is divided into quadrants with shaded regions to indicate different trade-offs. The green area represents methods that achieve best in both metrics. The blue and light pink areas highlight methods that emphasize one metric but have sacrificed the other. The red area shows methods with both metrics are low, making these methods less desirable. By comparing the positions of red and blue points, it can be observed the relative performance of different methods. Blue points often appear in the green quadrant, suggesting a favorable balance between the metrics. Each data point in the figure represents a specific method or configuration. Red points indicate state-of-the-art (SOTA) methods, while blue points represent the optiMAC model. The labels adjacent to the points (e.g., ProMAC and Agg MAC) correspond to different methods, and the accompanying values denote the tag-to-message ratio (TMR). For instance, a TMR of 0.9 implies that for every 10 messages, there are 9 tags, highlighting the balance between security and overhead. As shown in Figure 9, the Aggregate MAC achieves high security and the highest goodput. On the other hand, optiMAC offers enhanced security with only a slight reduction in goodput (Pareto optimal), demonstrating the trade-off between these factors. This flexibility allows our model to adapt to the specific requirements of various applications, even under adversarial scenarios.

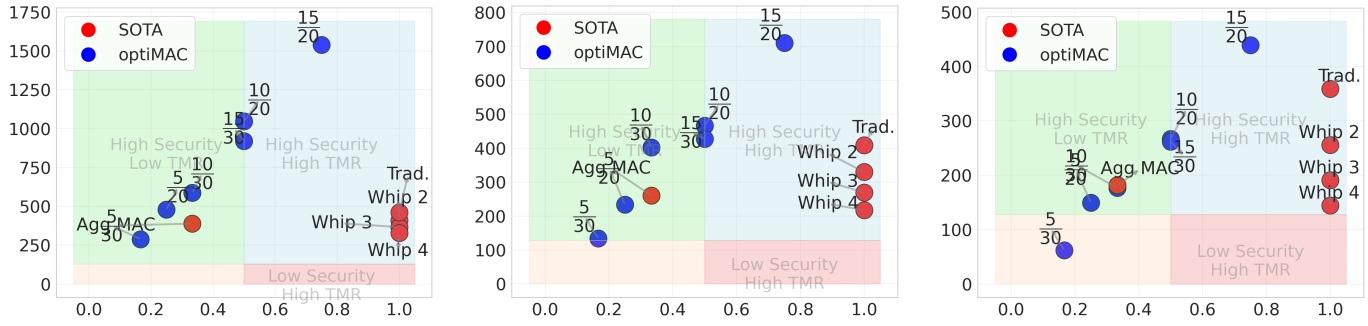
C. Security and Goodput vs TMR

Figure 10 illustrates the performance of different MAC schemes (SOTA, OptiMAC, Agg. MAC, and various ProMAC versions) under varying attack rates (0.1, 0.2, and 0.3). Each subgraph plots the "Tag-bits per Message" (y-axis) against the "TMR" $\frac{n}{m}$ (x-axis), providing insights into the trade-offs between security and overhead.

We note that the WiFi results are not shown due to space limitations, as they exhibit trends nearly identical to the 5G results. Specifically, the relative performance ordering of the schemes, the security–overhead trade-offs, and the impact of increasing attack rates remain consistent across both network settings. Therefore, to avoid redundancy and improve clarity, we focus on presenting the 5G results, which are representative of both environments.

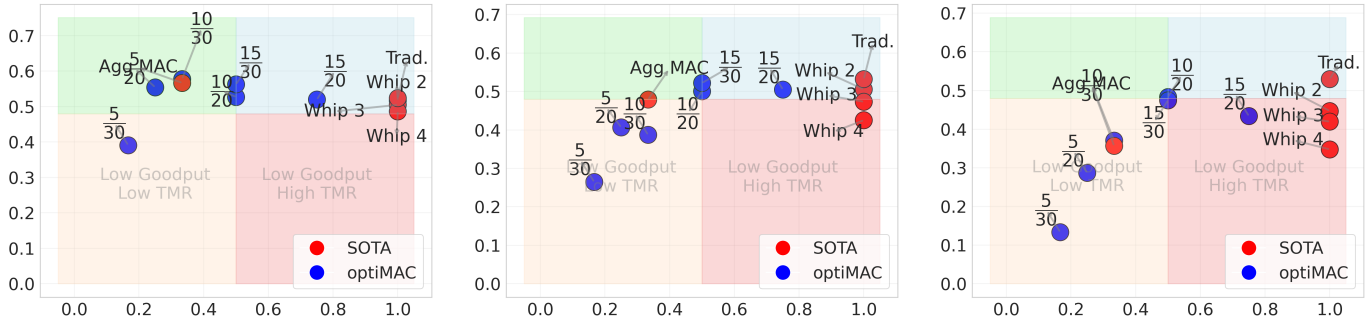
OptiMAC (depicted in blue) consistently achieves a balance between high security and low TMR, placing it in the "High Security, Low TMR" (green) region and satisfying Goal G2. It appears at lower tag-to-message ratios ($\frac{n}{m}$), indicating that it achieves strong security without requiring large redundancy, thereby reducing overhead. This demonstrates its efficiency compared to other schemes.

The SOTA solutions, shown in red, generally achieve high security but at higher TMR values, often placing them in the "High Security, High TMR" (blue) region, especially under higher attack rates. This indicates that while SOTA methods are effective, they incur significantly higher overhead compared to OptiMAC.



(a) Attack rate 0.1 (b) Attack rate 0.2 (c) Attack rate 0.3

Fig. 10: TbpM (y-axis) vs. TMR (x-axis) of 5G experiments with 512-bit tags (256 or higher TbpM is high security). State-of-the-art (SOTA) includes traditional, progressive, and aggregate MAC vs. the optiMAC.



(a) Attack rate 0.1 (b) Attack rate 0.2 (c) Attack rate 0.3

Fig. 11: Goodput (y-axis) vs. TMR (x-axis) of 5G experiments where more than 80% of Traditional MAC's Goodput is considered high. State-of-the-art (SOTA) includes traditional, progressive, and aggregate MAC vs. the optiMAC.

The Agg. MAC, when configured with a TMR of $\frac{1}{3}$, can achieve high security with low TMR under moderate attack rates, placing it in favorable regions of the plot. However, its performance degrades under higher attack rates (e.g., 0.3), where it transitions into the "Low Security, Low TMR" region, indicating insufficient robustness. In contrast, ProMAC variants (with dependency areas of 2, 3, and 4 without tag length adjustment) generally occupy the "Low Security, High TMR" or "High Security, High TMR" regions. While they can provide improved security over Agg. MAC, they do so at the cost of significantly increased overhead.

As the attack rate increases (from 0.1 to 0.3), all schemes exhibit a decrease in tag-bits per message, reflecting the need for increased redundancy to maintain security in more adversarial conditions.

Although not shown, the WiFi experiments follow the same trends, with differences in absolute values due to system parameters. In particular, WiFi exhibits lower tag-bits per message compared to 5G due to shorter tag lengths (32 vs. 64 bytes) and smaller payload sizes, as summarized in Table III.

Furthermore, by observing the trend of the OptiMAC configurations (blue points) in the goodput analysis (Figure 11), we identify a characteristic curve indicating an optimal operating region where goodput is maximized while maintaining required security levels (Goal G1). Selecting configurations near this peak enables an effective balance between security and efficiency under varying attack conditions.

XI. Conclusion

This work presents OptiMAC, a framework for optimizing integrity verification in lossy and adversarial communication channels. OptiMAC addresses the trade-off between security and efficiency in MAC schemes by introducing a dependency-graph representation and a mixed-integer linear programming formulation for tag-to-message assignment.

Unlike traditional and heuristic-based approaches, OptiMAC moves beyond fixed or ad hoc configurations. While ProMAC established a general framework, it did not provide a systematic method for determining key parameters such as tag generation, update rules, and assignment strategies. OptiMAC fills this gap by modeling assignments as a dependency graph and optimizing them mathematically, ensuring configurations are tailored to the operating environment.

Evaluations in WiFi and 5G scenarios show that OptiMAC achieves higher effective security while operating at lower tag-to-message ratios than prior schemes. These results demonstrate the framework's practicality for energy-sensitive and mission-critical applications, where efficiency and robustness must coexist.

References

[1] ETSI TS 133 501 V17.5.0 (2022-05) - 5G; Security architecture and procedures for 5G System (3GPP TS 33.501 version 17.5.0 Release 17). Technical report, ETSI, 2022. Accessed: 2024-05-02.

[2] anonym anonymus. OptiMAC. 8 2025.

[3] Frederik Armknecht, Paul Walther, Gene Tsudik, Martin Beck, and Thorsten Strufe. Promacs: Progressive and resynchronizing macs for continuous efficient authentication of message streams. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, CCS '20, page 211–223, New York, NY, USA, 2020. Association for Computing Machinery.

[4] Mihir Bellare, Roch Guérin, and Phillip Rogaway. Xor macs: New methods for message authentication using finite pseudorandom functions. In Don Coppersmith, editor, *Advances in Cryptology — CRYPT0' 95*, pages 15–28, Berlin, Heidelberg, 1995. Springer Berlin Heidelberg.

[5] Dan Boneh, Craig Gentry, Ben Lynn, and Hovav Shacham. Aggregate and verifiably encrypted signatures from bilinear maps. In *Advances in Cryptology—EUROCRYPT 2003: International Conference on the Theory and Applications of Cryptographic Techniques, Warsaw, Poland, May 4–8, 2003 Proceedings 22*, pages 416–432. Springer, 2003.

[6] Colin Boyd, Britta Hale, Stig Frode Mjølsnes, and Douglas Stebila. From stateless to stateful: Generic authentication and authenticated encryption constructions with application to tls. In *Cryptographers' Track at the RSA Conference*, pages 55–71. Springer, 2016.

[7] D. Dolev and A. Yao. On the security of public key protocols. *IEEE Transactions on Information Theory*, 29(2):198–208, 1983.

[8] Rosario Gennaro and Pankaj Rohatgi. How to sign digital streams. In *Advances in Cryptology - CRYPTO '97, 17th Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 1997, Proceedings*, volume 1294 of *Lecture Notes in Computer Science*, pages 180–197. Springer, 1997.

[9] Kim Hartmann and Christoph Steup. The vulnerability of uavs to cyber attacks - an approach to the risk assessment. In *2013 5th International Conference on Cyber Conflict (CYCON 2013)*, pages 1–23, 2013.

[10] Jonathan Katz and Andrew Y Lindell. Aggregate message authentication codes. In *Cryptographers' Track at the RSA Conference*, pages 155–169. Springer, 2008.

[11] Vladimir Kolesnikov, Wonsuck Lee, and Junhee Hong. Mac aggregation resilient to dos attacks. In *2011 IEEE International Conference on Smart Grid Communications (SmartGridComm)*, pages 226–231. IEEE, 2011.

[12] Vladimir Kolesnikov, Wonsuck Lee, and Junhee Hong. Mac aggregation resilient to dos attacks. In *2011 IEEE International Conference on Smart Grid Communications (SmartGridComm)*, pages 226–231. IEEE, 2011.

[13] He Li, Vireshwar Kumar, Jung-Min Jerry Park, and Yaling Yang. Cumulative message authentication codes for resource-constrained networks. In *2020 IEEE Conference on Communications and Network Security (CNS)*, pages 1–9, 2020.

[14] Steve Lu, Rafail Ostrovsky, Amit Sahai, Hovav Shacham, and Brent Waters. Sequential aggregate signatures and multisignatures without random oracles. In *Advances in Cryptology-EUROCRYPT 2006: 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques, St. Petersburg, Russia, May 28-June 1, 2006. Proceedings 25*, pages 465–485. Springer, 2006.

[15] Anna Lysyanskaya, Silvio Micali, Leonid Reyzin, and Hovav Shacham. Sequential aggregate signatures from trapdoor permutations. Cryptology ePrint Archive, Paper 2003/091, 2003. https://eprint.iacr.org/2003/091.

[16] Di Ma and Gene Tsudik. Forward-secure sequential aggregate authentication. In *2007 IEEE Symposium on Security and Privacy (SP'07)*, pages 86–91. IEEE, 2007.

[17] National Institute of Standards and Technology. The galois/counter mode of operation (gcm) update, May 2007. Accessed: 2024-11-14.

[18] National Institute of Standards and Technology. Public comments on sp 800-38b, October 2024. Accessed: 2024-11-14.

[19] Dennis K. Nilsson, Ulf E. Larson, and Erland Jonsson. Efficient in-vehicle delayed data authentication based on compound message authentication codes. In *2008 IEEE 68th Vehicular Technology Conference*, pages 1–5, 2008.

[20] oai. Openairinterface - 5g software alliance for democratising wireless innovation. https://github.com/OPENAIRINTERFACE/openairinterface 5g, 2024. Accessed: 2024-11-10.

[21] open5gs. Open5gs - a free and open-source 5g core network implementation. https://github.com/open5gs/open5gs, 2024. Accessed: 2024-11-10.

[22] Dan Petro and David Vargas. Badge of shame: Breaking into secure facilities with osdp. https://www.blackhat.com/us-23/briefings/schedule/#badge-of-shame-breaking-into-secure-facilities-with-osdp-32762, August 2023. Presented at Black Hat USA 2023.

[23] Hossein Pirayesh and Huacheng Zeng. Jamming attacks and anti-jamming strategies in wireless networks: A comprehensive survey. *IEEE communications surveys & tutorials*, 24(2):767–809, 2022.

[24] Jackson Schmandt, Alan T. Sherman, and Nilanjan Banerjee. Minimac: Raising the bar for vehicular security with a lightweight message authentication protocol. *Vehicular Communications*, 9:188–196, 2017.

[25] Eric Wagner, Jan Bauer, and Martin Henze. Take a bite of the reality sandwich: Revisiting the security of progressive message authentication codes. In *Proceedings of the 15th ACM Conference on Security and Privacy in Wireless and Mobile Networks*, WiSec '22, page 207–221, New York, NY, USA, 2022. Association for Computing Machinery.

[26] Eric Wagner, Martin Serror, Klaus Wehrle, and Martin Henze. When and how to aggregate message authentication codes on lossy channels? In *International Conference on Applied Cryptography and Network Security*, pages 241–264. Springer, 2024.

[27] Jiarong Xing, Sophia Yoo, Xenofon Foukas, Daehyeok Kim, and Michael K. Reiter. On the criticality of integrity protection in 5g fronthaul networks. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 4463–4479, Philadelphia, PA, August 2024. USENIX Association.